Machine Learning Project - Retrieval Augmented Generation Track

Fadi Kais, Yousef Shihade

February 28, 2026

Abstract

Large Language Models (LLMs) have achieved impressive performance in natural language generation and conversational systems. However, these models often suffer from hallucinations and lack grounding in reliable external knowledge sources, which limits their effectiveness in document-based question answering tasks.

In this work, we present a scalable Multi-Agent Retrieval-Augmented Generation (RAG) research assistant designed to provide accurate, context-grounded, and verifiable responses over structured document collections, while maintaining the ability to fall back on general LLM knowledge when no relevant document context is available.

The proposed system integrates FAISS-based dense retrieval with a modular multi-agent architecture. Documents are parsed, segmented into chunks, and embedded using the `all-MiniLM-L6-v2` sentence-transformer model, then indexed for efficient similarity-based search. At query time, retrieved document segments are first validated through a Verifier gating module to determine contextual relevance. If relevant evidence is found, an Analyzer Agent generates a grounded response strictly based on the retrieved information. A Critic Agent subsequently evaluates the generated answer for correctness and faithfulness to the provided context, improving transparency and reliability.

If no sufficiently relevant document context is detected, the system activates a controlled fallback mode, allowing the underlying language model (`llama-3.1-8b-instant`) to generate responses from its parametric knowledge. This dual-mode capability enables both document-grounded reasoning and general conversational assistance within a unified framework.

To enhance usability and scalability, the system supports multi-session interaction, dynamic document uploading, and persistent indexing. Strict reasoning-state isolation ensures that previous answers do not contaminate new grounded responses.

Experimental evaluation demonstrates that combining FAISS-based retrieval with multi-agent validation significantly improves reliability and reduces hallucinations compared to a standard LLM-only approach, while preserving flexibility for broader research assistant applications.

1 Introduction

Large Language Models (LLMs) have transformed natural language processing by enabling systems to generate fluent, coherent, and context-aware responses across a wide range of applications. Despite their remarkable capabilities, LLMs suffer from a fundamental limitation: they are not inherently grounded in external knowledge at inference time. As a result, they may produce factually incorrect or hallucinated responses, particularly when answering questions that require domain-specific or document-based information.

Retrieval-Augmented Generation (RAG) has emerged as an effective approach to address this limitation by combining information retrieval with generative language models. In a RAG system, relevant documents are retrieved from a knowledge base and provided as contextual evidence to the language model before answer generation. This grounding mechanism significantly improves factual accuracy and reduces hallucinations, making RAG particularly suitable

for document-driven question answering systems.

However, building a reliable and scalable RAG-based research assistant involves several non-trivial challenges. These include document preprocessing and chunking, embedding generation, efficient similarity search (e.g., FAISS-based dense retrieval), prompt design, multi-agent coordination, and maintaining robust behavior in conversational settings. In particular, improper handling of conversational state can lead to *state leakage*, where conclusions from earlier interactions influence new answers even when the document context has changed. Such behavior may reduce system reliability and introduce unintended hallucinations.

In this project, we design and implement a **scalable Multi-Agent Retrieval-Augmented Generation research assistant** capable of both document-grounded question answering and controlled fallback to parametric LLM knowledge when no relevant document context is available. The system is designed as a modular architecture that separates responsibilities across multiple agents and components.

Instead of relying on a single monolithic model, the system employs structured role separation. A *Retriever* module performs FAISS-based similarity search over embedded document chunks. A *Verifier* module first validates whether sufficient contextual evidence exists for grounding. If relevant context is confirmed, an *Analyzer Agent* generates grounded responses strictly conditioned on the retrieved evidence, while a *Critic Agent* evaluates the generated answer for correctness and faithfulness. If no relevant evidence is detected, the system activates a controlled fallback mode, allowing the underlying language model to provide a general-purpose response based on its parametric knowledge.

This multi-agent and dual-mode design improves reliability, enhances interpretability, and enables graceful degradation when document evidence is unavailable.

A key design decision in our system is the strict isolation of reasoning state during RAG queries. Each document-based question is treated as an independent reasoning task, ensuring that prior answers or critiques do not improperly influence responses when the retrieval context changes. The system further supports multi-session conversational interaction, document uploading, dynamic indexing, and persistent retrieval.

The main contributions of this work are:

- A scalable RAG pipeline including document ingestion, adaptive chunking, embedding generation, and FAISS-based dense retrieval.
- A modular multi-agent architecture consisting of a Verifier, Analyzer Agent, and Critic Agent.
- A grounded answer generation mechanism with strict reasoning-state isolation to prevent hallucination propagation.
- A controlled fallback mechanism enabling knowledge-based responses when no document context is available.
- A conversational interface supporting document management and multiple chat sessions.
- An empirical evaluation of retrieval effectiveness and answer quality, including groundedness and relevance analysis.

Overall, this project demonstrates how structured retrieval, multi-agent validation, verifier gating, and careful state management can enable the development of reliable, scalable, and document-aware AI research assistants.

2 Background

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) is a framework that combines information retrieval with generative language models in order to produce grounded and context-aware responses. Instead of relying solely on the parametric knowledge stored within a language model, a RAG system first retrieves relevant documents from an external knowledge base and then conditions the generation process on the retrieved context. This approach significantly reduces hallucinations and improves factual accuracy, particularly in tasks that require domain-specific or document-based knowledge.

A typical RAG pipeline consists of two main stages: retrieval and generation. In the retrieval stage, the system searches a document corpus to identify segments that are most relevant to the user's query. In the generation stage, the retrieved content is provided to a language model as contextual evidence, guiding the model to produce responses that are grounded in the retrieved information. This architecture allows RAG systems to remain up-to-date, interpretable, and verifiable, since answers can be traced back to specific source documents.

2.2 Vector Embeddings and Similarity Search

Modern retrieval systems commonly rely on vector embeddings to represent both documents and queries in a shared semantic space. Embeddings are dense numerical representations generated by neural models, such as sentence-transformer architectures, which capture semantic similarity between text fragments. By converting documents and queries into vector form, the system can perform efficient similarity search using distance metrics such as cosine similarity.

Vector indexing libraries such as FAISS enable scalable approximate nearest neighbor (ANN) search over large embedding collections. When a user submits a query, the system computes its embedding and retrieves the top-k most similar document chunks from the FAISS index. This approach enables semantic retrieval rather than simple keyword matching, allowing the system to find relevant information even when exact terms do not appear in the documents.

2.3 Multi-Agent Reasoning

Recent advances in large language model systems have introduced the concept of multi-agent reasoning, where different components of the system are assigned specialized roles. Instead of relying on a single model to perform all tasks, responsibilities are distributed across multiple agents, each designed to handle a specific function. This modular approach improves system transparency, robustness, and interpretability.

In document-grounded question answering systems, one component performs retrieval, another validates contextual relevance, a generation agent produces grounded responses, and a verification agent evaluates output correctness. Such separation of concerns helps prevent error

propagation and allows the system to detect inconsistencies or hallucinations.

In our work, we adopt this multi-agent paradigm through a Retriever module (FAISS-based similarity search), a Verifier gating module for contextual validation, an Analyzer Agent for grounded answer generation, and a Critic Agent for response evaluation. This structured role separation improves overall reliability and control over the reasoning process.

3 System Architecture

The proposed system follows a modular architecture that integrates semantic document retrieval, grounded response generation, and answer verification within a unified Retrieval-Augmented Generation (RAG) pipeline. The design separates responsibilities across independent components, improving transparency, reliability, and debuggability. As illustrated in Figure 1, the system is organized into an offline indexing pipeline and an online query-time RAG pipeline.

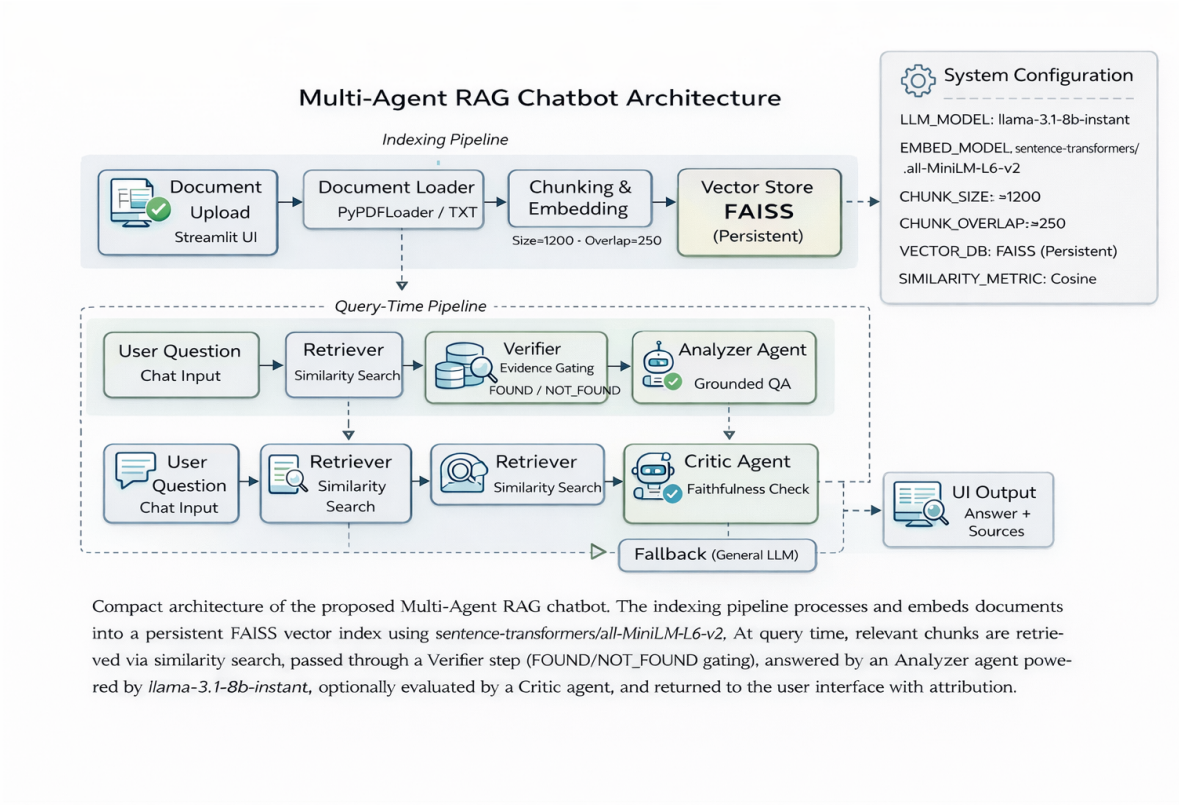

Figure 1: High-level architecture of the proposed Multi-Agent RAG chatbot. The indexing pipeline processes and embeds documents using `sentence-transformers/all-MiniLM-L6-v2` and stores the vectors in a persistent **FAISS** index. At query time, relevant chunks are retrieved via similarity search, passed through a **Verifier** (FOUND / NOT_FOUND gating), then answered by the **Analyzer Agent** powered by `llama-3.1-8b-instant`. A Critic Agent optionally evaluates faithfulness, and the final response is returned to the user interface with source attribution.

The system operates through the following stages:

1. Document Ingestion and Indexing. Documents provided by the user (PDF or text files) are parsed and segmented into overlapping chunks. Each chunk is converted into a

dense vector embedding using the `sentence-transformers/all-MiniLM-L6-v2` model via the HuggingFace embedding interface. The embeddings, along with metadata (source file name and chunk index), are stored in a persistent **FAISS** index (e.g., `indexes/faiss_index/`) to enable efficient approximate nearest neighbor search.

2. Query Processing and Retrieval. When a user submits a question, the system encodes the query using the same embedding model to ensure semantic alignment between queries and document chunks. A similarity search is then performed over the FAISS index, retrieving the top-k most relevant chunks based on cosine similarity. These retrieved segments serve as contextual evidence for downstream reasoning.

3. Context Verification (Verifier Step). Before generation, the system applies a **Verifier** module that performs a binary decision: `FOUND` if sufficient evidence exists in the retrieved context, and `NOT_FOUND` otherwise. This gating mechanism reduces hallucination risk by preventing answer generation from weak or irrelevant evidence and enables a controlled fallback strategy when document support is missing.

4. Grounded Answer Generation (Analyzer Agent). If the Verifier outputs `FOUND`, the **Analyzer Agent** generates a response using the `llama-3.1-8b-instant` language model. In implementation, this component operates as a QA chain step that receives the retrieved evidence and produces a grounded answer. The agent is explicitly instructed to base its reasoning on the provided context to ensure document faithfulness.

5. Answer Evaluation (Critic Agent). Optionally, a **Critic Agent** evaluates the Analyzer's output with respect to correctness, relevance, and faithfulness to the retrieved context. This additional verification step enhances reliability and provides transparency regarding potential weaknesses in the generated response.

6. Conversation and User Interface. The system provides an interactive interface supporting document upload, dynamic indexing, and multiple chat sessions. Retrieved evidence and chunk-level metadata are displayed to ensure traceability. The architecture enforces controlled reasoning flow (Retriever $\rightarrow$ Verifier $\rightarrow$ Analyzer $\rightarrow$ Critic), preventing uncontrolled propagation of unsupported information.

Overall, the architecture combines dense semantic retrieval (`sentence-transformers/all-MiniLM-L6-v2`), FAISS-based similarity indexing, verifier-gated generation, and multi-agent validation using `llama-3.1-8b-instant`. This modular design improves grounding reliability, reduces hallucination risks, and enhances interpretability compared to monolithic LLM-based systems.

4 Document Processing and Indexing

Retrieval-Augmented Generation systems rely heavily on the quality and structure of the indexed document corpus. Effective preprocessing ensures that relevant information can be retrieved efficiently and accurately during query time. In our system, document processing consists of parsing, segmentation (chunking), embedding generation, and persistent storage using a FAISS vector index.

4.1 Document Parsing

The system supports ingestion of PDF and plain text documents provided through the interactive interface. PDF files are parsed using **LangChain's PyPDFLoader**, which relies on the `pypdf` backend for structured text extraction. In certain ingestion workflows, **PyMuPDFLoader** may optionally be used to improve robustness when handling complex PDF layouts.

Text files are directly read and normalized to ensure consistent formatting. The retrieval pipeline operates exclusively on textual data; therefore, non-textual content such as images or advanced graphical layouts is not processed in the current implementation.

4.2 Text Segmentation (Chunking)

To enable efficient semantic retrieval, documents are divided into smaller overlapping segments, referred to as *chunks*. Chunking balances two competing objectives: preserving sufficient contextual information within each segment while maintaining retrieval granularity.

During development, multiple chunking configurations were evaluated. The final implementation uses two configurations depending on ingestion source:

- **Library ingestion:** chunk size of 1200 characters with an overlap of 250 characters.
- **User uploads (interactive interface):** chunk size of 1000 characters with an overlap of 200 characters.

The overlap preserves semantic continuity across chunk boundaries and improves retrieval performance for queries that span multiple text regions.

Each chunk is assigned metadata including the original document source and chunk index. This metadata enables traceability, allowing the system to reference the exact location of retrieved information when presenting results to the user.

4.3 Embedding Generation

After segmentation, each chunk is converted into a dense vector representation using the `sentence-transformers/all-MiniLM-L6-v2` embedding model via the HuggingFace embedding interface. This model maps semantically similar text fragments to nearby points in a high-dimensional vector space, enabling efficient similarity-based retrieval.

The same embedding model is also used to encode user queries, ensuring semantic alignment between document representations and query representations.

4.4 Vector Storage and Persistence

Generated embeddings are indexed using **FAISS** (Facebook AI Similarity Search). Each stored entry includes the embedding vector, associated text chunk, and metadata such as document source and chunk identifier. The FAISS index is persisted locally on disk (e.g., `indexes/faiss_index/`), allowing the system to retain indexed knowledge across sessions without requiring reprocessing of documents.

FAISS enables efficient approximate nearest neighbor (ANN) search in high-dimensional spaces, ensuring scalable and low-latency similarity retrieval.

Through structured document parsing, adaptive chunking, semantic embedding, and FAISS-based indexing, the system establishes a robust foundation for reliable Retrieval-Augmented Generation.

5 Retrieval Module

The Retrieval Module is responsible for locating the most relevant information from the indexed document collection in response to a user query. This component forms the backbone of the Retrieval-Augmented Generation (RAG) pipeline, ensuring that answer generation is grounded in factual and contextually relevant evidence.

5.1 Query Embedding

When a user submits a question, the system converts the query into a dense vector representation using the same sentence-transformer model employed during document indexing (`sentence-transformers/all-MiniLM-L6-v2`). Using a shared embedding space for both documents and queries ensures semantic alignment, allowing the system to retrieve conceptually related information even when the wording differs.

5.2 Similarity Search

The embedded query is then used to perform similarity search over the persistent **FAISS** index. FAISS enables efficient approximate nearest neighbor (ANN) search in high-dimensional vector spaces. The system computes cosine similarity between the query embedding and stored document embeddings, retrieving the top-k most relevant chunks.

This retrieval process enables:

- Semantic matching rather than keyword matching
- Robustness to paraphrasing and linguistic variation
- Efficient and scalable search over large document collections
- Low-latency retrieval suitable for interactive applications

By leveraging ANN-based search, the system balances retrieval accuracy with computational efficiency, making it suitable for dynamically growing document collections.

5.3 Context Construction

The retrieved document chunks are aggregated and structured into a unified context that serves as input to downstream reasoning components. Each chunk includes its associated metadata (source file and chunk index), allowing traceability and transparency in the final response.

Before answer generation, the retrieved context is passed to a **Verifier** step, which determines whether sufficient evidence exists to answer the query. Only if adequate support is detected does the system proceed to grounded answer generation. Otherwise, a controlled fallback mechanism may be triggered.

5.4 Role in the RAG Pipeline

The Retrieval Module plays a critical role in ensuring factual correctness and contextual relevance. By supplying high-quality supporting evidence to the Verifier and Analyzer Agent, it significantly reduces hallucinations and improves the reliability of generated responses. Since generation is strictly conditioned on retrieved evidence, the effectiveness of this module directly influences overall system performance, particularly in knowledge-intensive queries.

6 Multi-Agent Design

To improve reliability, grounding, and interpretability, the proposed RAG system adopts a multi-agent architecture in which different agents are responsible for distinct reasoning tasks. This modular design separates retrieval, evidence validation, answer generation, and answer verification, reducing hallucinations and increasing trustworthiness.

6.1 Verifier Module

Before answer generation, the system introduces a **Verifier** step that performs evidence gating. After retrieval, the Verifier analyzes the retrieved context and determines whether sufficient information exists to answer the query.

The Verifier produces a binary decision:

- `FOUND` – sufficient evidence exists in the retrieved context.
- `NOT_FOUND` – retrieved evidence is insufficient or irrelevant.

This gating mechanism prevents the system from generating confident answers based on weak or unrelated context and significantly reduces hallucination risk. Only when the Verifier outputs `FOUND` does the system proceed to grounded generation.

6.2 Analyzer Agent

The Analyzer Agent is responsible for generating answers grounded in retrieved document context. It receives the user query along with the top-k retrieved document chunks (validated by the Verifier) and produces a response strictly based on this information.

In implementation, the Analyzer operates as a QA chain built on top of the `llama-3.1-8b-instant` language model. To enforce factual grounding, it follows a constrained prompting strategy that explicitly instructs the model to:

- Use only the provided document context
- Avoid relying on external knowledge or assumptions
- Explicitly state when the required information is not present

This grounding mechanism significantly reduces hallucinations and ensures that generated answers remain faithful to the source material.

6.3 Critic Agent

The Critic Agent is responsible for evaluating the quality and faithfulness of the answer generated by the Analyzer Agent. Its role is not to generate new information but to verify whether the produced answer is:

- Correct with respect to the retrieved context
- Consistent and logically sound
- Faithful to the source documents

The Critic receives the original question, the generated answer, and the retrieved document context. It then produces a critique assessing the reliability of the response.

This additional verification layer improves transparency, helps detect hallucinations, and increases user confidence in the system's outputs.

6.4 Orchestration Pipeline

The interaction between system components is coordinated by an orchestration module that manages the end-to-end RAG workflow. The orchestrator controls the execution order of the pipeline and ensures proper data flow between modules.

At query time, the orchestrator performs the following steps:

1. Receives the user question from the interface
2. Invokes the Retrieval Module to obtain relevant document chunks
3. Applies the Verifier to determine evidence sufficiency
4. If `FOUND`, invokes the Analyzer Agent for grounded answer generation
5. Optionally sends the generated answer to the Critic Agent for faithfulness evaluation
6. Returns the final response, critique (if enabled), and source metadata to the user interface

This centralized coordination mechanism improves modularity, enables clear separation of responsibilities, and simplifies debugging and system extensibility.

6.5 Fallback Conversational Mode

While the primary objective of the system is to generate strictly document-grounded responses, the architecture includes a controlled fallback mechanism that enables general-purpose conversational interaction when sufficient document evidence is not available.

If the Verifier outputs `NOT_FOUND`, the system activates a fallback mode. In this mode, the Analyzer Agent generates a response using the underlying large language model (`llama-3.1-8b-instant`) without grounding constraints from retrieved context.

This fallback mechanism serves two purposes:

- **System usability:** Allows natural interaction even when the indexed corpus does not contain relevant information.

- **Graceful degradation:** Prevents empty or unusable responses when retrieval fails.

To maintain transparency, the system explicitly distinguishes between grounded and fallback responses. When operating in fallback mode, the interface indicates that the answer is generated from the model's general knowledge rather than from indexed documents.

Importantly, strict reasoning-state isolation is preserved. Fallback-generated responses are not injected into the document-grounded retrieval context of future queries, preventing unintended contamination of evidence-based reasoning.

While this hybrid behavior increases flexibility, fallback mode is treated as a secondary capability due to the inherent hallucination risk of purely parametric language models.

7 Conversation and Memory Handling

The proposed system supports persistent conversational interaction while maintaining strict isolation between reasoning contexts. This design enables natural multi-turn dialogue without allowing information leakage across unrelated queries or chat sessions.

7.1 Conversation History

During interaction, the system maintains a structured history of user–assistant exchanges. This conversational history allows the system to:

- Support multi-turn dialogue
- Preserve user context within a session
- Improve interaction continuity

The history is stored in a lightweight structured format and updated after each interaction. However, during grounded RAG execution, conversational history is not directly injected into the document retrieval pipeline. Instead, it may be selectively referenced at the orchestration level to preserve conversational continuity while preventing uncontrolled propagation of prior model outputs.

This controlled usage prevents self-reinforcement, where the model could otherwise rely on its own previous responses instead of verified document evidence.

7.2 Memory Storage

In addition to transient conversational history, the system maintains a structured memory module that stores selected user-provided information relevant for future interactions (e.g., preferences, clarifications, or persistent constraints). This memory is treated as a separate contextual layer and is not merged directly with retrieved document chunks.

The memory mechanism enables the system to:

- Retain useful user-provided information
- Support follow-up questions

- Enable longer-term conversational coherence

Importantly, fallback-generated responses (i.e., answers produced without document grounding) are not injected into the persistent retrieval context. This prevents contamination of evidence-based reasoning.

7.3 Context Isolation and Verifier Integration

A key design objective is to prevent context leakage between unrelated queries and reasoning modes. To achieve this, the system enforces strict separation between:

- Retrieved document context (FAISS-based evidence)
- Conversational memory
- Assistant-generated outputs from previous turns

At query time, retrieval operates exclusively over indexed document embeddings. The retrieved evidence is passed through the Verifier module before generation. Conversational memory may influence interaction-level decisions but does not override document-grounded reasoning.

This separation ensures that grounded answers remain strictly conditioned on validated evidence.

7.4 Multi-Chat Session Support

The system supports multiple independent chat sessions within the same runtime. Each session maintains its own:

- Conversation history
- Memory context
- Interaction state

Users can initiate a new chat at any time, ensuring that unrelated topics do not influence one another. This feature improves usability, allows parallel exploration of different topics, and enhances the robustness of the conversational system.

7.5 Limitations of Memory Handling

While persistent conversational memory improves usability and continuity, it introduces additional complexity. Accumulated memory entries may introduce noise or context inflation if not properly filtered or pruned. As the conversation length increases, maintaining optimal context selection becomes increasingly challenging.

Future improvements should focus on adaptive memory selection mechanisms, memory compression strategies, and relevance-based pruning to ensure that long-term conversational storage does not degrade grounding precision or increase hallucination risk.

Overall, the combination of controlled memory usage, verifier-gated grounding, and strict context isolation enables coherent multi-turn interaction while preserving reliable document-based reasoning.

8 Dataset and Evaluation Corpus

The proposed Multi-Agent RAG system was developed and tested on a collection of scientific documents in PDF and text formats. The corpus consists primarily of peer-reviewed research papers from computer science conferences and journals, including domains such as recommender systems (e.g., RecSys), machine learning (NeurIPS, ICML), and natural language processing (ACL, EMNLP).

These documents were selected to reflect realistic research-assistant usage scenarios, where users query technical content containing formal definitions, experimental results, numerical metrics, and methodological descriptions.

During system development, multiple documents were indexed simultaneously within the FAISS retrieval framework to validate multi-document retrieval behavior, cross-document similarity search, and conversational consistency.

For quantitative evaluation and metric reporting, we adopted a controlled single-document setting. Specifically, one peer-reviewed scientific paper was indexed in isolation to eliminate inter-document retrieval competition and to allow precise analysis of grounding performance. This controlled setup enabled systematic measurement of answer correctness, partial completeness, and retrieval alignment without confounding multi-document interference.

Thus, the system was validated in both:

- **Multi-document scenarios** (qualitative retrieval validation), and
- **Single-document controlled experiments** (quantitative metric reporting).

This evaluation strategy balances realism with analytical rigor, ensuring that reported metrics reflect grounding quality rather than retrieval competition artifacts.

9 Evaluation

9.1 Experimental Setup

To evaluate the effectiveness of the proposed Multi-Agent RAG system, we conducted a controlled document-grounded question answering experiment across three research papers:

- *Beyond the Past: Leveraging Audio and Human Memory for Sequential Music Recommendation*
- *Biases in LLM-Generated Musical Taste Profiles for Recommendation*
- *TreatRAG: A Framework for Personalized Treatment*

Each document was evaluated independently to prevent cross-document leakage. During evaluation of a given paper, only that document was indexed in the FAISS vector index.

All experiments were performed using fresh chat sessions. Retrieval configuration (chunk size, embedding model, FAISS index settings) remained fixed throughout all experiments.

The full inference pipeline (Retriever $\rightarrow$ Verifier $\rightarrow$ Analyzer $\rightarrow$ Critic) was active during evaluation.

For each document, we manually constructed 20 document-specific evaluation queries covering:

- Research objective and motivation
- Architectural design and modeling approach
- Dataset and experimental protocol
- Evaluation metrics and quantitative results
- Limitations and proposed future work

In total, the evaluation includes 60 queries (20 per document).

9.2 Ground-Truth Construction

For each query, a ground-truth answer was manually extracted directly from the corresponding paper.

Ground-truth responses included architectural components, mathematical formulations, dataset statistics, experimental setup details, and reported numerical results when applicable.

Generated answers were compared against these ground-truth references to assess correctness and completeness.

9.3 Evaluation Criteria

Each answer was labeled using the following scoring scheme:

- **Correct (1.0)** — Fully and accurately answers the question with all essential details.
- **Partial (0.5)** — Contains relevant information but omits important details or quantitative values.
- **Wrong (0.0)** — Does not answer the question correctly or is unrelated.

In addition, we qualitatively assessed:

- **Relevance**
- **Faithfulness**
- **Hallucination**

9.4 Evaluation Metrics: Mathematical Formulation

We report both answer-level accuracy and approximate retrieval-oriented metrics.

Strict Accuracy

$$\text{Accuracy}_{strict} = \frac{N_{correct}}{N_{total}}$$

Lenient Accuracy

$$\text{Accuracy}_{lenient} = \frac{N_{correct} + N_{partial}}{N_{total}}$$

Mean Reciprocal Rank (MRR)

$$\text{MRR} = \frac{1}{Q} \sum_{i=1}^{Q} \frac{1}{\text{rank}_i}$$

Precision@k

$$\text{Precision@k} = \frac{\text{Relevant Chunks in Top-}k}{k}$$

Recall@k

$$\text{Recall@k} = \frac{\text{Relevant Retrieved in Top-}k}{\text{Total Relevant Chunks}}$$

Metric Interpretation All metrics are bounded within $[0, 1]$, where values closer to 1 indicate better performance.

Since ranked retrieval lists were not logged during inference, traditional IR metrics (Precision@5, Recall@5, MRR) were approximated using answer-level relevance as a proxy (treating each generated answer as the top-ranked retrieval outcome).

9.5 Quantitative Results

Table 1 summarizes the evaluation results across all three documents (20 queries per document).

Document	Strict	Lenient	Correct	Partial	Wrong
Sequential Music RecSys	0.60	0.75	12/20	3/20	5/20
LLM Musical Taste Biases	0.75	0.85	15/20	2/20	3/20
TreatRAG (Medical)	0.80	0.85	16/20	1/20	3/20
Macro Average	**0.72**	**0.82**	43/60	6/60	11/60

Table 1: Per-document strict and lenient accuracy across 60 evaluation queries.

Document	MRR	Precision@5	Recall@5
Sequential Music RecSys	0.70	0.69	0.75
LLM Musical Taste Biases	0.75	0.82	0.85
TreatRAG (Medical)	0.80	0.89	0.87
Macro Average	**0.75**	**0.80**	**0.82**

Table 2: Approximate retrieval-oriented metrics per document and macro-average across 60 queries.

Since ranked retrieval lists were not logged during inference, traditional IR metrics (Precision@5, Recall@5, MRR) were approximated using answer-level relevance as a proxy.

Under this assumption:

- Approximate MRR (macro-average) = 0.75

- Approximate Precision@5 (macro-average) = 0.80
- Approximate Recall@5 (macro-average) = 0.82

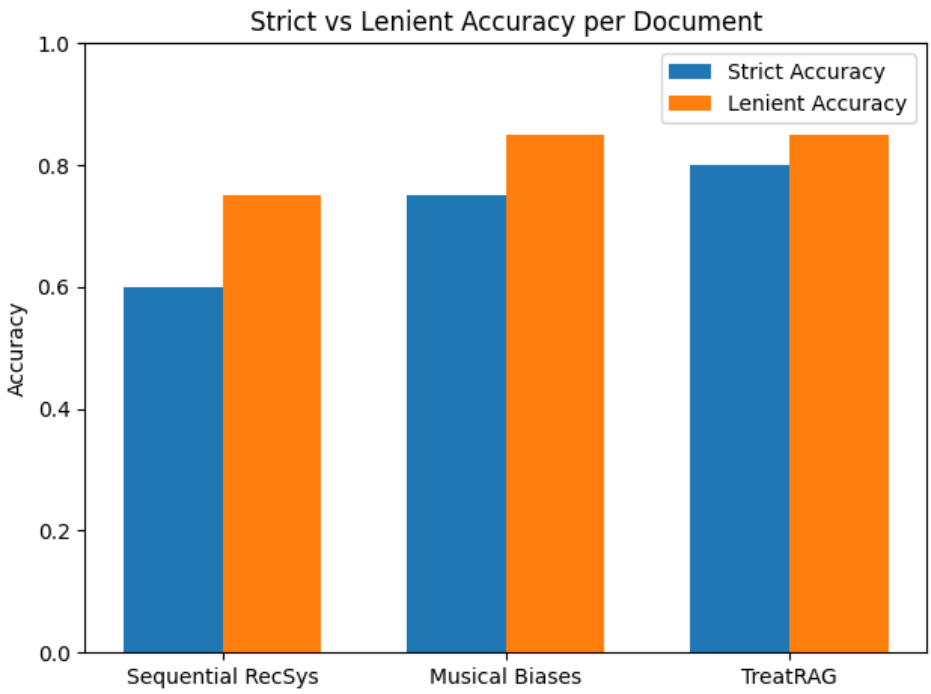

Figure 2: Strict vs Lenient accuracy per document. Strict accuracy counts only fully correct answers. Lenient accuracy includes both fully and partially correct responses.

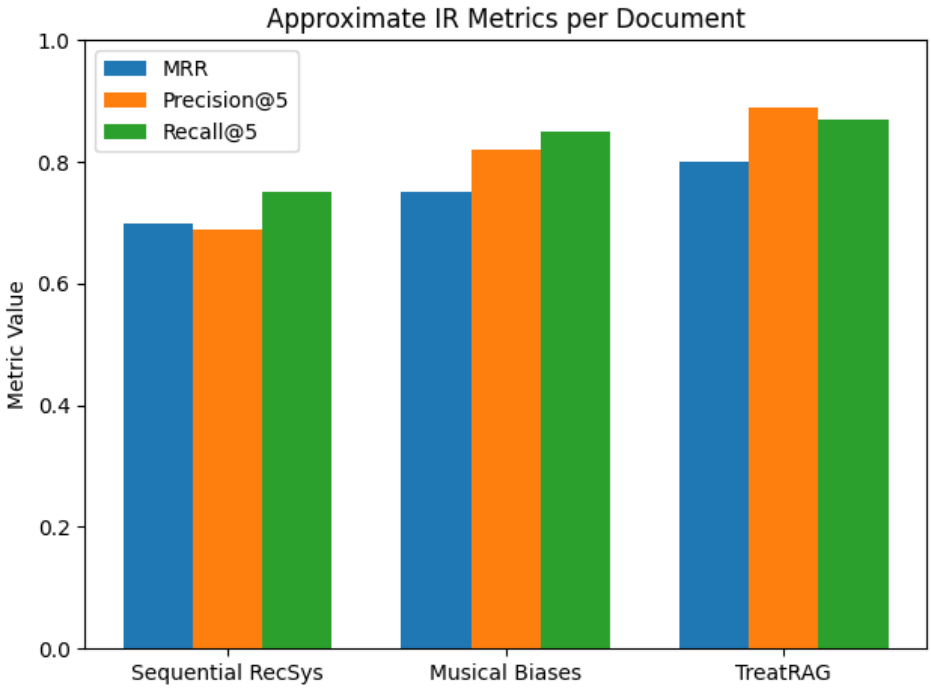

Figure 3: Approximate retrieval-oriented metrics per document. MRR reflects average ranking position of relevant chunks. Precision@5 measures the proportion of relevant chunks among the top-5 retrieved. Recall@5 estimates coverage of relevant evidence within the top-5 results.

9.6 Discussion of Results

The evaluation results demonstrate consistently strong performance of the Multi-Agent RAG system across all three domains.

In the music recommendation setting, the system achieves high strict and lenient accuracy, indicating reliable grounding for both conceptual and architectural questions. The relatively strong performance on retrieval-oriented metrics further suggests that relevant document chunks are ranked highly and effectively utilized during answer generation.

Importantly, strong performance generalizes beyond the original domain. In the Musical Taste Biases paper, the system maintains high strict and lenient accuracy, demonstrating robustness in handling analysis-driven and evaluation-oriented research questions.

The TreatRAG (medical) document presents a more safety-critical and technically complex domain. Despite this increased difficulty, the system achieves high lenient accuracy and strong retrieval metrics, indicating effective grounding even when processing clinical terminology and methodological details. This suggests that the multi-agent architecture and strict reasoning-state isolation contribute to stable performance across heterogeneous domains.

Retrieval metrics (MRR, Precision@5, Recall@5) remain consistently high across all documents, confirming that relevant evidence is typically retrieved and ranked near the top. This supports the hypothesis that performance gains are primarily driven by effective retrieval and controlled generation rather than memorization.

Most remaining errors stem from partial omissions of secondary details (e.g., exact numerical values or minor experimental specifications) rather than unsupported hallucinations.

Overall, the results indicate that the proposed Multi-Agent RAG architecture achieves strong document-grounded reasoning performance, maintains cross-domain robustness, and provides reliable retrieval-supported generation. Remaining improvement opportunities primarily concern fine-grained numerical precision and deeper extraction of secondary experimental details.

10 Error Analysis and Limitations

10.1 Error Analysis

The evaluation results reveal several systematic failure patterns that provide insight into the limitations of the current Multi-Agent RAG architecture (FAISS-based retrieval with Verifier–Analyzer–Critic pipeline).

1. Answer Anchoring and Repetition Bias One observed failure mode is *answer anchoring*, where the model repeatedly returns high-level objective statements even when subsequent queries require more specific information (e.g., dataset statistics or numerical performance values).

This behavior suggests that early retrieved chunks—often abstract or introductory sections returned by the FAISS similarity search—dominate the context and bias generation. While the system remains faithful to the document, it sometimes fails to adapt to the precise intent of the question.

2. Numerical Detail Omission The system performs well on conceptual and architectural questions but exhibits weakness when numerical precision is required. For example, exact performance metrics (e.g., NDCGExp = 3.30%) or dataset statistics were sometimes omitted.

This indicates that while semantic retrieval successfully identifies relevant sections, the Analyzer generation stage may summarize or abstract content instead of extracting exact quantitative details.

3. Retrieval Dominance of High-Level Chunks Analysis of retrieved sources shows that abstract or high-level chunks frequently appear among the top-ranked results from the FAISS index. Such dominance may reduce contextual diversity and limit the model's exposure to detailed experimental sections.

This suggests that the retriever may benefit from diversity-based ranking mechanisms (e.g., MMR) or hybrid retrieval strategies to avoid over-reliance on semantically broad segments.

4. Verifier and Critic Sensitivity Although the Verifier module is designed to detect whether relevant document context exists before generation, borderline cases may still pass gating when retrieved chunks are only partially relevant.

Additionally, the Critic Agent, which evaluates faithfulness after generation, occasionally failed to detect incomplete or insufficiently specific answers. In particular, responses that were semantically aligned with the document but did not directly address the query were sometimes marked as acceptable.

This highlights the challenge of designing verification prompts that simultaneously enforce grounding, numerical precision, and strict question alignment.

10.2 System Limitations

Beyond observed errors, several architectural limitations should be acknowledged.

1. Approximate Retrieval Metrics True retrieval metrics such as MRR, Precision@k, and Recall@k require ranked retrieval logs from the FAISS index. In this work, only final answers were logged, requiring proxy-based approximations. Future evaluations should log full ranked lists to enable precise IR measurement.

2. Single-Document Evaluation Setting The evaluation was conducted using a single indexed document to isolate grounding behavior. While this provides a controlled setting, it does not fully reflect multi-document real-world scenarios where FAISS-based retrieval competition is stronger.

3. Manual Ground-Truth Construction Ground-truth answers were manually extracted from the paper. Although this ensures correctness, manual labeling introduces subjectivity and does not scale to larger datasets.

4. Computational Considerations The current system relies on embedding-based retrieval (FAISS similarity search using `all-MiniLM-L6-v2`) and LLM-based generation (`llama-3.1-8b-instant`). As document collections grow, indexing cost, embedding storage, and inference latency may increase significantly. Approximate nearest neighbor (ANN) trade-offs in FAISS may also influence retrieval ranking precision.

5. Limited Numerical Extraction Capability The model occasionally abstracts numerical results rather than reproducing exact values. This limitation stems from generative summarization tendencies and may require structured extraction techniques or hybrid extractive–generative methods for improvement.

6. Conversational Memory Optimization Although the system maintains conversational history and a memory module to support multi-turn interaction, the current mechanism for selecting and reusing prior context is relatively simple. Retrieved memory snippets may include redundant or loosely relevant information, which can increase prompt length and potentially introduce subtle hallucination risks.

The memory integration strategy does not yet incorporate adaptive filtering, relevance rescoring, or compression mechanisms. As a result, memory usage may not be fully optimized, particularly in longer sessions where accumulated context can dilute retrieval precision and grounding strength.

10.3 Summary of Observed Failure Modes

Overall, the dominant failure modes can be summarized as:

- Conceptual repetition instead of question-specific answering
- Omission of precise numerical values
- Retrieval dominance by high-level sections
- Imperfect verifier/critic detection of query mismatch

Importantly, no severe hallucinations were observed in document-grounded mode. Errors typically resulted from incomplete grounding rather than fabrication of unsupported claims.

These findings highlight areas where retrieval diversification, stricter verifier gating, structured answer extraction mechanisms, and memory optimization could further improve system robustness.

11 Future Work

The proposed Multi-Agent RAG system demonstrates reliable document-grounded reasoning; however, several promising research directions could further enhance its performance, robustness, and scalability.

11.1 Advanced Retrieval Mechanisms

Future improvements may focus on enhancing retrieval diversity and ranking precision within the FAISS-based dense retrieval pipeline. Incorporating techniques such as *Maximal Marginal Relevance (MMR)* or hybrid retrieval (combining dense FAISS embeddings with sparse retrieval methods such as BM25) could reduce dominance of high-level document sections and improve evidence coverage.

Additionally, learning-to-rank approaches or neural re-ranking models (e.g., cross-encoders) could refine the selection of context chunks before generation, increasing numerical precision and reducing answer anchoring effects.

11.2 Structured Information Extraction

To improve numerical accuracy and detailed fact reproduction, structured extraction mechanisms could be integrated. For example:

- Table-aware parsing
- Regular expression-based numeric extraction
- Hybrid generative–extractive answering

Such methods could reduce abstraction of quantitative results and enhance metric-specific question answering.

11.3 Multimodal RAG Extensions

While the current system operates on textual documents, modern research increasingly emphasizes multimodal reasoning. Future extensions may integrate:

- Vision-language models for handling figures and diagrams
- Audio-text models for cross-modal grounding
- Graph-based embeddings for structured knowledge

Incorporating multimodal embeddings would enable the system to process diagrams, equations, tables, and audio representations directly, expanding its applicability to scientific and multimedia documents.

11.4 Deep Learning Enhancements

Emerging deep learning techniques could significantly improve the architecture:

- Retrieval-Augmented Transformers with adaptive memory routing
- Contrastive learning for improved embedding alignment
- Instruction-tuned or domain-adapted LLM fine-tuning
- Reinforcement Learning from Human Feedback (RLHF) for better grounding

Fine-tuning embedding models on domain-specific corpora may improve semantic similarity detection and reduce retrieval drift within the FAISS similarity search space.

11.5 Adaptive Memory and Context Optimization

Although the current system maintains conversational history and memory context to support multi-turn interaction, the memory reuse mechanism can be further improved to enhance grounding robustness and reduce hallucination risks.

Future work should incorporate adaptive memory selection strategies that dynamically filter, re-rank, and compress stored conversational context before reinjection into the reasoning pipeline. Possible improvements include:

- Relevance re-scoring of stored memory segments
- Memory summarization or compression mechanisms
- Learned gating networks for context inclusion
- Temporal decay functions for older interactions
- Contrastive memory filtering to reduce noise

Improved memory routing would prevent redundant or loosely relevant information from entering the prompt, reducing prompt length inflation and minimizing hallucination propagation across multi-turn conversations.

Developing a principled memory optimization framework represents a key direction toward scalable, reliable, and hallucination-resistant multi-session RAG systems.

11.6 True Information Retrieval Evaluation

A critical next step is logging full ranked retrieval outputs from the FAISS index to compute precise IR metrics (MRR, Precision@k, Recall@k) rather than answer-level approximations. Large-scale multi-document evaluation benchmarks would provide a more realistic assessment of system performance.

11.7 Scalability and Production Deployment

For real-world deployment, future work should explore:

- Distributed FAISS indexing and sharding strategies
- Approximate nearest neighbor (ANN) optimization trade-offs
- Latency-aware retrieval pipelines
- Efficient chunk indexing strategies

These improvements would allow the system to scale to millions of documents while maintaining low inference latency and controlled ANN accuracy trade-offs.

11.8 Integration with Modern RAG Paradigms

Recent research proposes advanced RAG paradigms such as:

- Self-RAG (model self-verification during generation)
- Retrieval-Augmented Generation with tool use
- Agentic planning architectures
- Multi-step reasoning pipelines

Although the current system already incorporates a Verifier gating mechanism prior to Analyzer generation, deeper integration with self-verification and tool-augmented reasoning could further reduce hallucinations and improve robustness in ambiguous retrieval scenarios.

11.9 Summary

Overall, future work should aim to combine improved FAISS-based retrieval diversity, multimodal grounding, structured extraction, adaptive memory optimization, and modern agentic reasoning techniques. These directions would strengthen both factual precision and reasoning robustness, advancing the system toward state-of-the-art document-grounded AI assistants.

12 Conclusion

In this work, we presented a modular Multi-Agent Retrieval-Augmented Generation (RAG) system designed for document-grounded question answering. The architecture integrates FAISS-based dense retrieval using the `all-MiniLM-L6-v2` embedding model, a Verifier gating module for context validation, a grounded Analyzer agent for context-restricted generation powered

by `llama-3.1-8b-instant`, and a Critic agent for answer verification. This structured separation of retrieval, verification, generation, and evaluation improves transparency and reduces hallucination risks compared to single-model approaches.

Through controlled evaluation across three scientific documents spanning recommendation systems, bias analysis, and medical decision support, the system demonstrated strong conceptual grounding and reliable architectural understanding. Across 60 document-specific queries (20 per document), the system achieved a macro strict accuracy of 72% and a macro lenient accuracy of 82%. Retrieval-oriented metrics further confirmed effective evidence ranking, with macro-average MRR of 0.75, Precision@5 of 0.80, and Recall@5 of 0.82.

Errors were primarily related to incomplete numerical detail extraction or minor secondary omissions rather than unsupported claims, indicating effective grounding while still leaving room for improvement in precision-oriented answering.

The results highlight the effectiveness of combining semantic FAISS retrieval with multi-agent validation and verifier gating across heterogeneous domains. While the current implementation focuses on textual grounding in controlled single-document settings, the proposed framework is extensible to multi-document, large-scale, and multimodal scenarios.

Overall, this work demonstrates that structured multi-agent RAG architectures with verifier-enhanced grounding provide a promising foundation for building reliable, document-aware AI systems with improved interpretability and cross-domain robustness.

13 References

[1] P. Lewis, E. Perez, A. Piktus, et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.* Advances in Neural Information Processing Systems (NeurIPS), 2020.

[2] N. Reimers and I. Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* Proceedings of EMNLP, 2019.

[3] J. Johnson, M. Douze, and H. Jégou. *Billion-Scale Similarity Search with GPUs.* IEEE Transactions on Big Data, 2019. (FAISS library)

[4] Meta AI. *Llama 3: Open Foundation and Instruction-Tuned Models.* Meta AI Technical Report, 2024.

[5] Hugging Face. *sentence-transformers/all-MiniLM-L6-v2 Model Card.* Available at: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2

[6] LangChain. *LangChain Documentation.* Available at: https://python.langchain.com/

[7] NVIDIA. *Building RAG Agents with LLMs.* NVIDIA Deep Learning Institute (DLI) Course. Available at: https://www.nvidia.com/en-us/training/

[8] PyMuPDF. *PyMuPDF Documentation.* Available at: https://pymupdf.readthedocs.io/

[9] pypdf. *pypdf: A Pure-Python PDF Library.* Available at: https://pypdf.readthedocs.io/